

# Operating Systems (OS)

## Categories

Prof. Dr.-Ing. Jörg Abke



**TH Aschaffenburg**  
university of applied sciences

# Copyright Hinweise

© Jörg Abke

Dieses Werk ist urheberrechtlich geschützt. Alle Rechte, insbesondere Vervielfältigung, Verbreitung oder Übersetzung liegen beim Autor. Kein Teil des Werkes darf ohne Genehmigung des Autors, auch nicht für Unterrichtszwecke, fotokopiert oder durch andere Verfahren reproduziert oder verbreitet werden. Ebenfalls ist die Einspeicherung in EDV-Anlagen ohne Genehmigung des Autors nicht gestattet. Jeder Studierende, der an der entsprechenden Lehrveranstaltung des Autors an der Technischen Hochschule Aschaffenburg teilnimmt, ist berechtigt, einen Ausdruck dieses Werkes anzufertigen und für die Zwecke seines Studiums zu verwenden. Eine weitergehende Verwendung bedarf in jedem Falle der ausdrücklichen Zustimmung des Autors. Gebrauchsname, Handelsnamen, Warenbezeichnungen etc. werden in diesem Werk ohne Gewährleistung der freien Verwendbarkeit benutzt. Texte, Abbildungen, Programme und technische Angaben wurden sorgfältig erarbeitet. Trotzdem sind Fehler nicht völlig auszuschließen. Der Autor weist darauf hin, dass er für Fehlerfreiheit keine Gewähr und für eventuelle Folgen keine Haftung übernimmt.

# Classification of OS

## Categories

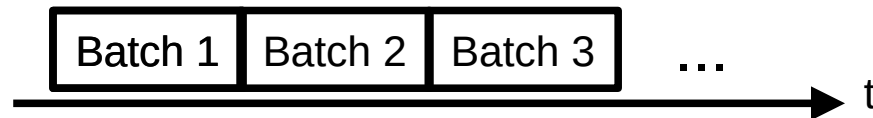
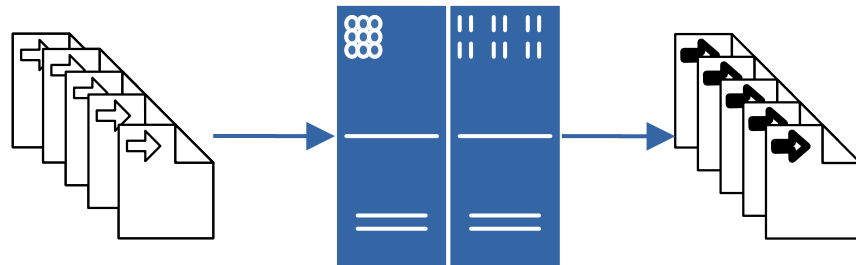
- **Interaction Mode**
  - Batch Processing / Batchbetrieb
  - Time Sharing / Dialogbetrieb
- **Single Tasking vs. Multitasking**
- **Single-User vs. Multi-User**
- **Target Hardware Memory Address Space**
  - 8, 16, 32, 64bit
- **Hard Real Time OS vs. Soft Real Time OS (RTOS)**
- **Architecture of RTOS**
  - Thin Kernel
  - Nano Kernel

# Batch Processing vs. Time Sharing

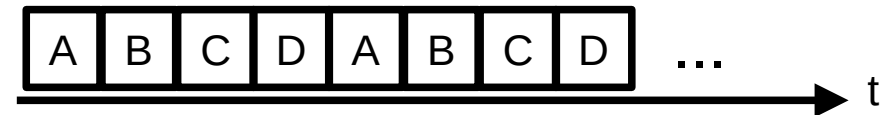
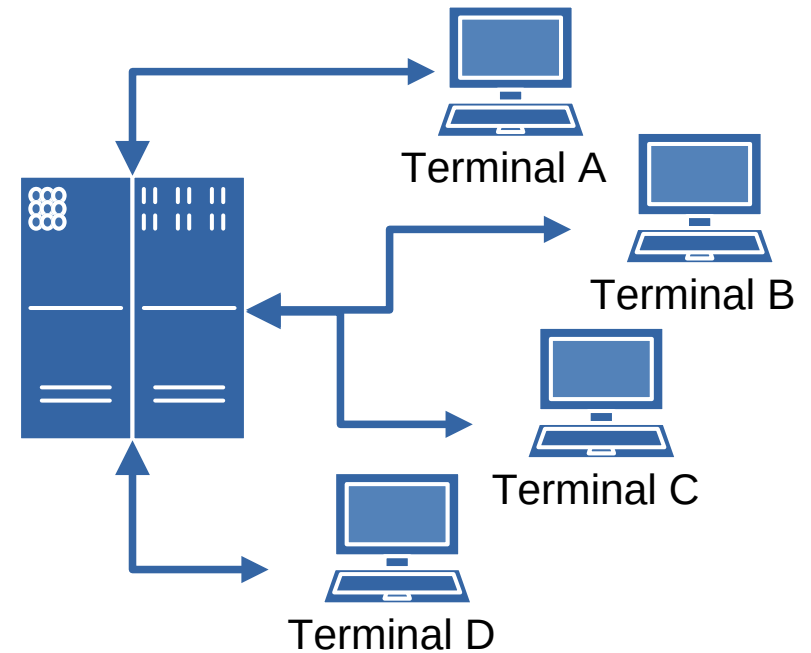


TH Aschaffenburg  
university of applied sciences

## Batch Processing

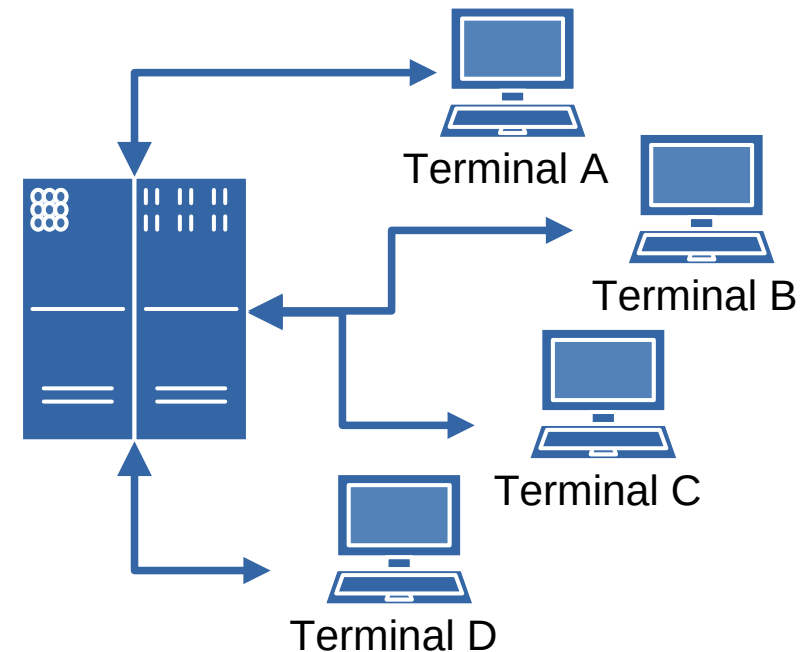
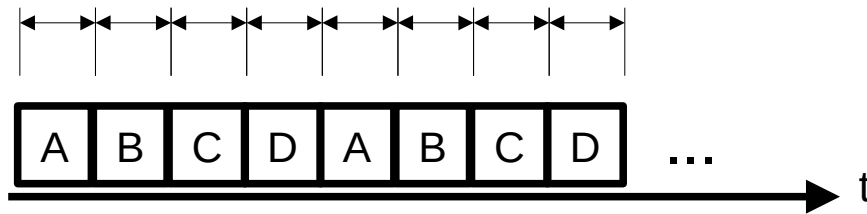


## Time Sharing / Interactive Mode



# Time Sharing Interactive Mode Details

- **Time Slices → Scheduling (→ later chapter)**



- **Multiple users can run multiple tasks**
- **Like any modern Desktop / Server Operating Systems**

# Some Terminology beforehand



TH Aschaffenburg  
university of applied sciences

## Process or task are synonymes

- Program in execution
- Used interchangeably, same meaning
- A Program is the term for a process from a user's perspective
- A Job
  - typically used in batch systems
  - Includes one or more programs

```
top - 14:37:58 up 5:43, 8 users, load average: 0.45, 0.36, 0.45
tasks: 186 total, 1 running, 184 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.1%us, 0.0%sy, 0.0%ni, 99.9%id, 0.0%wa, 0.0%st, 0.0%hi, 0.0%si
Mem: 4090688k total, 2530648k used, 1479832k free, 108868k buffers
Swap: 0k total, 0k used, 0k free, 0k cached
```

| PID   | USER  | PR | NI  | VIRT  | RES   | SHR  | S | %CPU | TIME+ | COMMAND                 |
|-------|-------|----|-----|-------|-------|------|---|------|-------|-------------------------|
| 1000  | root  | 20 | 0   | 122k  | 44k   | 27k  | S | 0    | 1.1   | 0:23.82 klogd           |
| 9154  | abike | 20 | 0   | 112k  | 35k   | 27k  | S | 0    | 0.0   | 0:01.26 dhcpd           |
| 3183  | abike | 20 | 0   | 1039k | 1012k | 44k  | S | 0    | 4.7   | 0:23.34 thunderbird     |
| 3044  | abike | 20 | 0   | 152k  | 30k   | 91k  | S | 0    | 1.0   | 0:48.24 plugin-containe |
| 2070  | abike | 20 | 0   | 357k  | 87k   | 44k  | S | 1    | 2.2   | 0:42.45 plasma-desktop  |
| 8199  | abike | 20 | 0   | 134k  | 53k   | 42k  | S | 1    | 1.4   | 0:08.76 ksmcde          |
| 7627  | avard | 20 | 0   | 441k  | 275k  | 100k | S | 0    | 0.1   | 0:04.31 aucti-daemon    |
| 2073  | abike | 20 | 0   | 273k  | 112k  | 81k  | S | 0    | 0.0   | 0:26.03 kryptguard      |
| 9489  | abike | 20 | 0   | 1027k | 75k   | 27k  | S | 0    | 2.0   | 0:00.99 java            |
| 9438  | root  | 20 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.05 kworker/0:1     |
| 9433  | root  | 20 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.19 kworker/virt    |
| 15752 | abike | 20 | 0   | 285k  | 116k  | 68k  | R | 0    | 0.0   | 0:00.05 top             |
| 1     | root  | 20 | 0   | 2632  | 200k  | 1212 | S | 0    | 0.1   | 0:00.00 init            |
| 2     | root  | 20 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 kthreadd        |
| 3     | root  | 20 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.10 ksoftirqd/0     |
| 4     | root  | 87 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 migration/0     |
| 7     | root  | 87 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.15 watchdog/0      |
| 21    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 ksm             |
| 24    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 kswapd0         |
| 25    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 ksm             |
| 26    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.04 sync_supers     |
| 27    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 kfs-default     |
| 28    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 kcongregat      |
| 29    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 kblock          |
| 30    | root  | 0  | -20 | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 ata_sff         |
| 31    | root  | 20 | 0   | 0     | 0     | 0    | S | 0    | 0.0   | 0:00.00 shubd           |

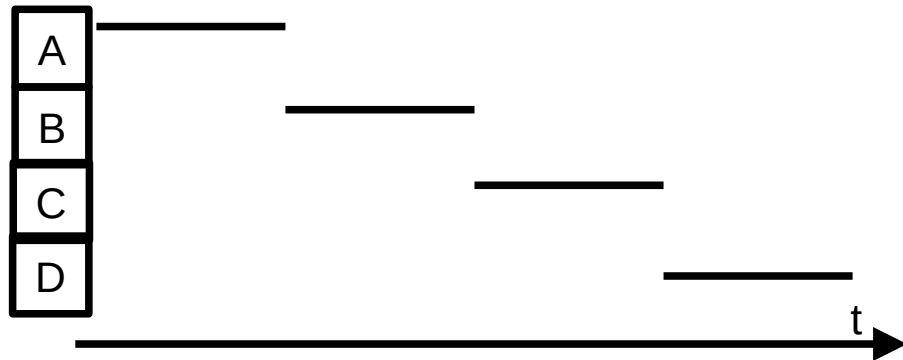
Screenshot of top

# Single Tasking

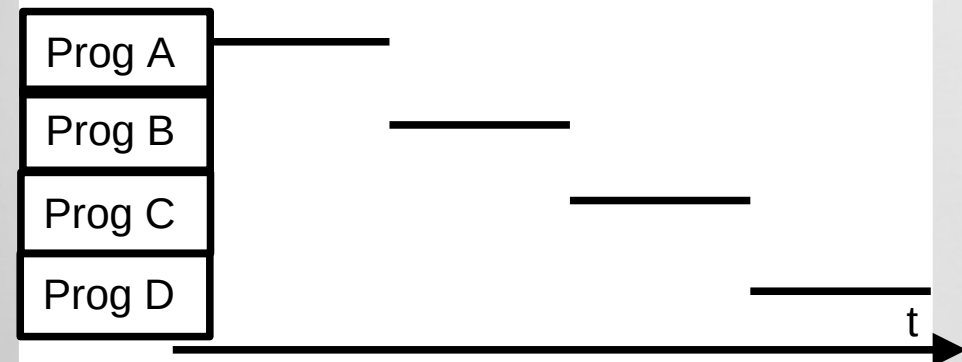


TH Aschaffenburg  
university of applied sciences

- A single CPU core can only execute one task another



- A single tasking system executes only one program at a time

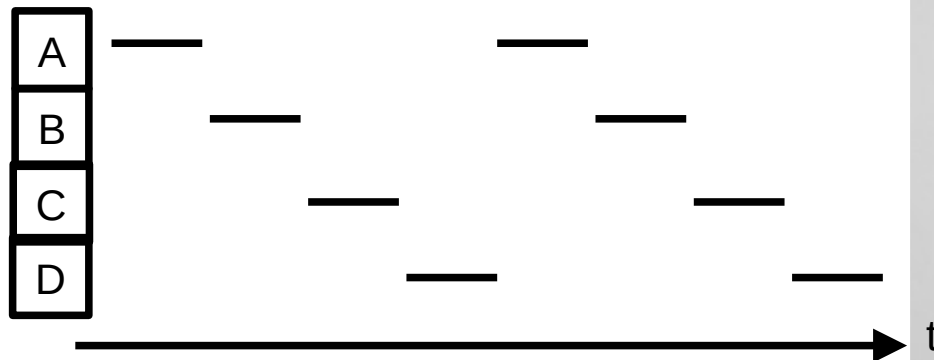


# Multitasking

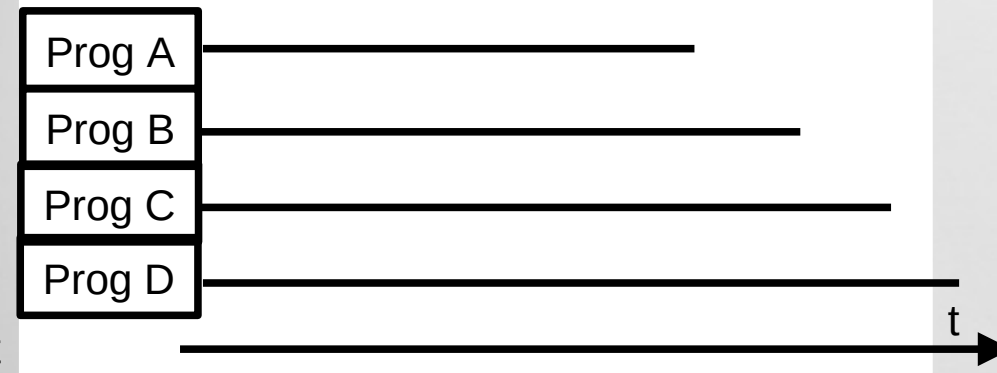


TH Aschaffenburg  
university of applied sciences

**A single CPU core can only execute one process at any point in time**



- Multiple programs seem to be executed at the same time from users' perspective (pseudo parallel)**





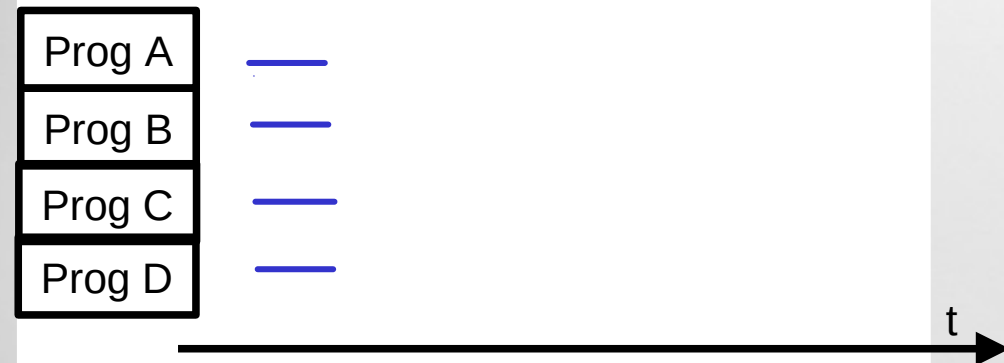
# Multitasking on Multiple CPU Cores

**Assume 4 CPU cores**  
**Depict the 4 tasks in the diagram**



Some concurrency in hardware (like memory or mass storage bottleneck, or other hardware (IO) which is only one time in a system)

- **Assume 4 CPU cores**
- **Depict the 4 programs in the diagram**

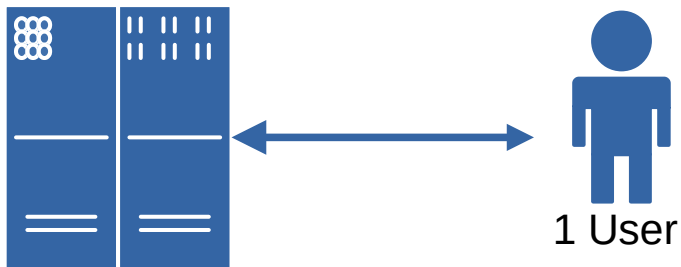


# Single User vs. Multi User



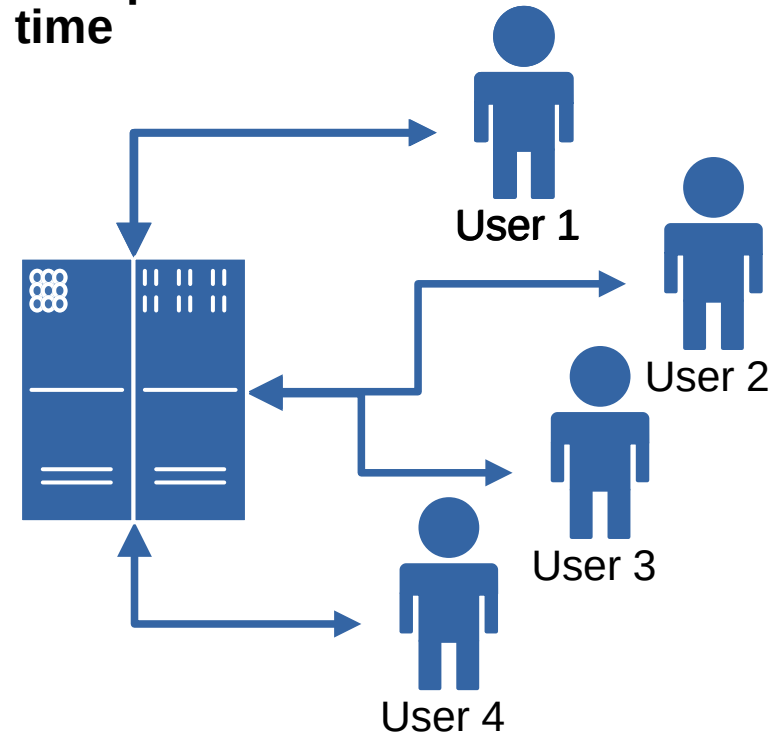
TH Aschaffenburg  
university of applied sciences

- A single user can use the system at any point in time



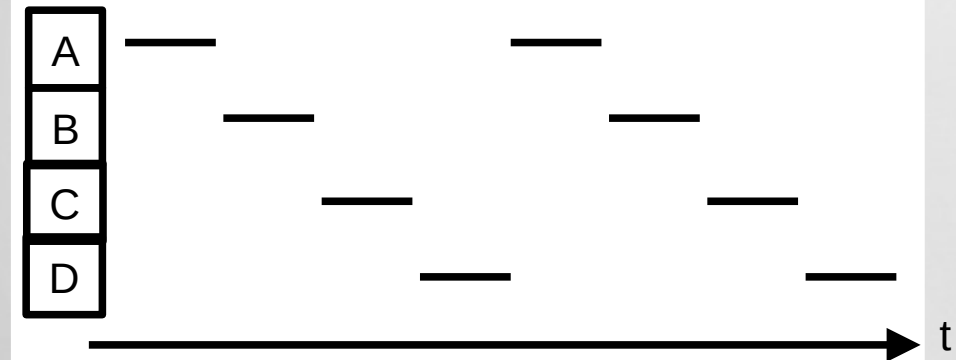
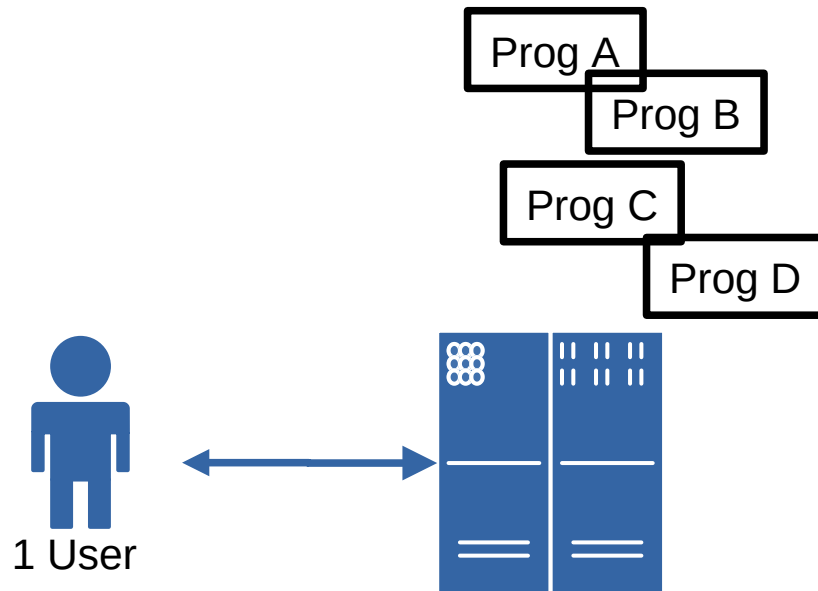
- Examples: MS-DOS, Microsoft Windows 3x/95/98, OS/2

- Multiple users use it at the same time



- Examples: Unices (plural of Unix), Microsoft NT Server and higher, Mac OS

# How about a Single User but Multi Tasking?



# Memory Address Space Categories with OS Examples



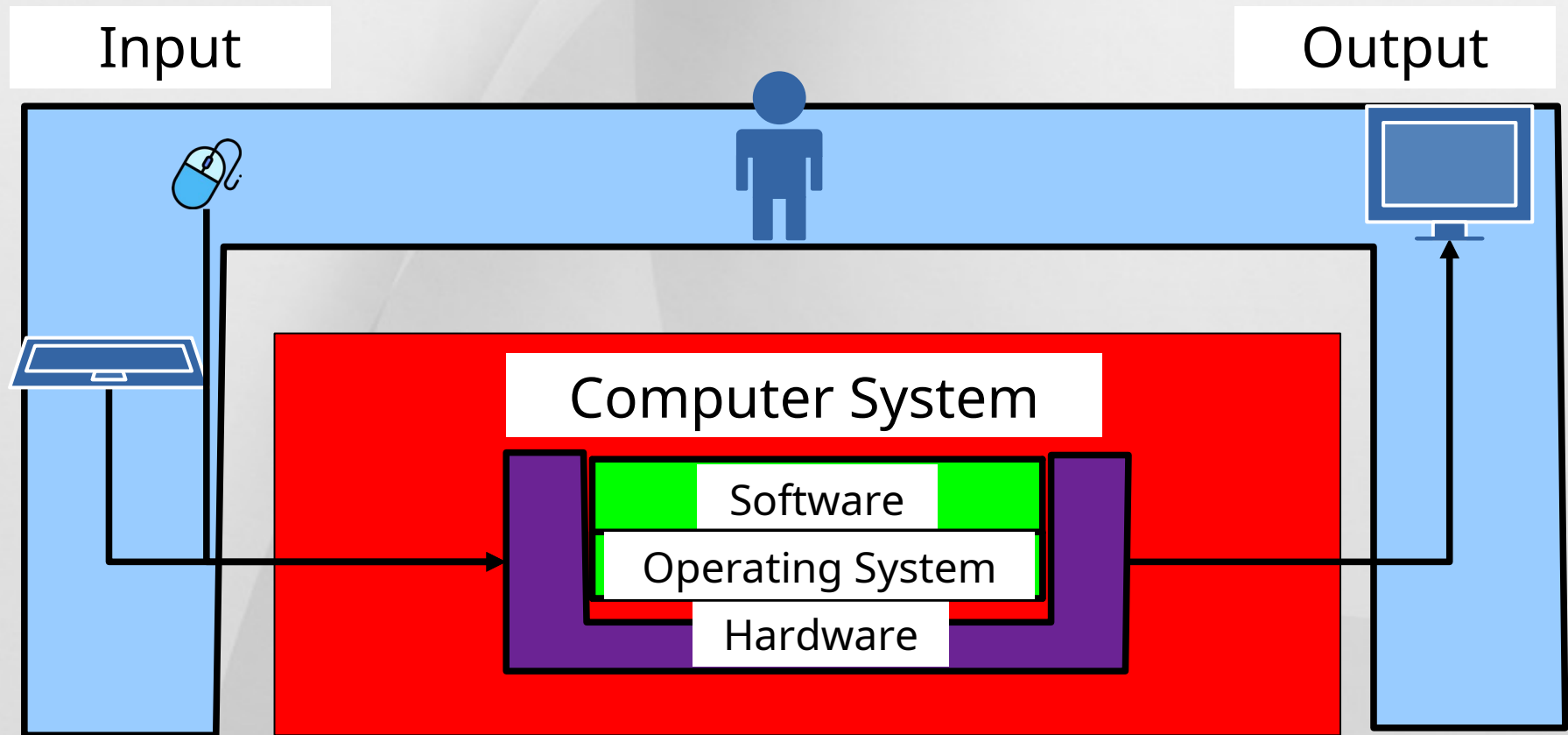
TH Aschaffenburg  
university of applied sciences

- **An 8 bit OS is able to manage  $2^8 = 256$  addresses**
- **A 16 bit OS  $2^{16} = 65536$  addresses**
- **A 32 bit OS  $2^{32}$  addresses**
  - Windows 95, 98, NT, Vista, 7,8,10
  - Linux (32 bit)
  - Mac OS X (up to 10.7)
- **A 64 bit OS  $2^{64}$  addresses**
  - Windows 7, 8, 10, 11 (64 bit)
  - Linux (64 bit)
  - Mac OS X (64 bit)

# Typical Computer System with User Interaction – Intro Real Time Systems



TH Aschaffenburg  
university of applied sciences

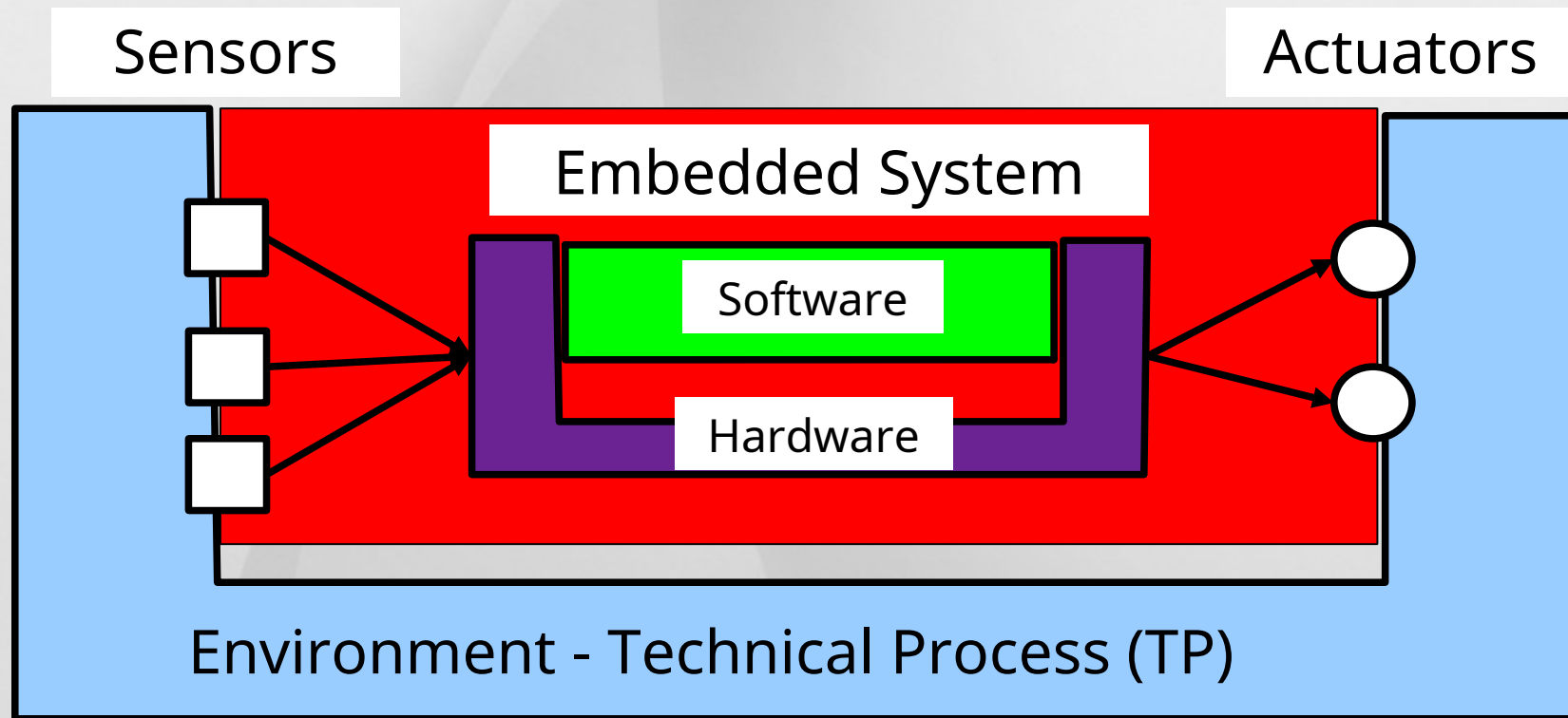


# Typical Computer System for Control

## - Embedded System – Intro Real Time Systems



TH Aschaffenburg  
university of applied sciences



request  $r_i$

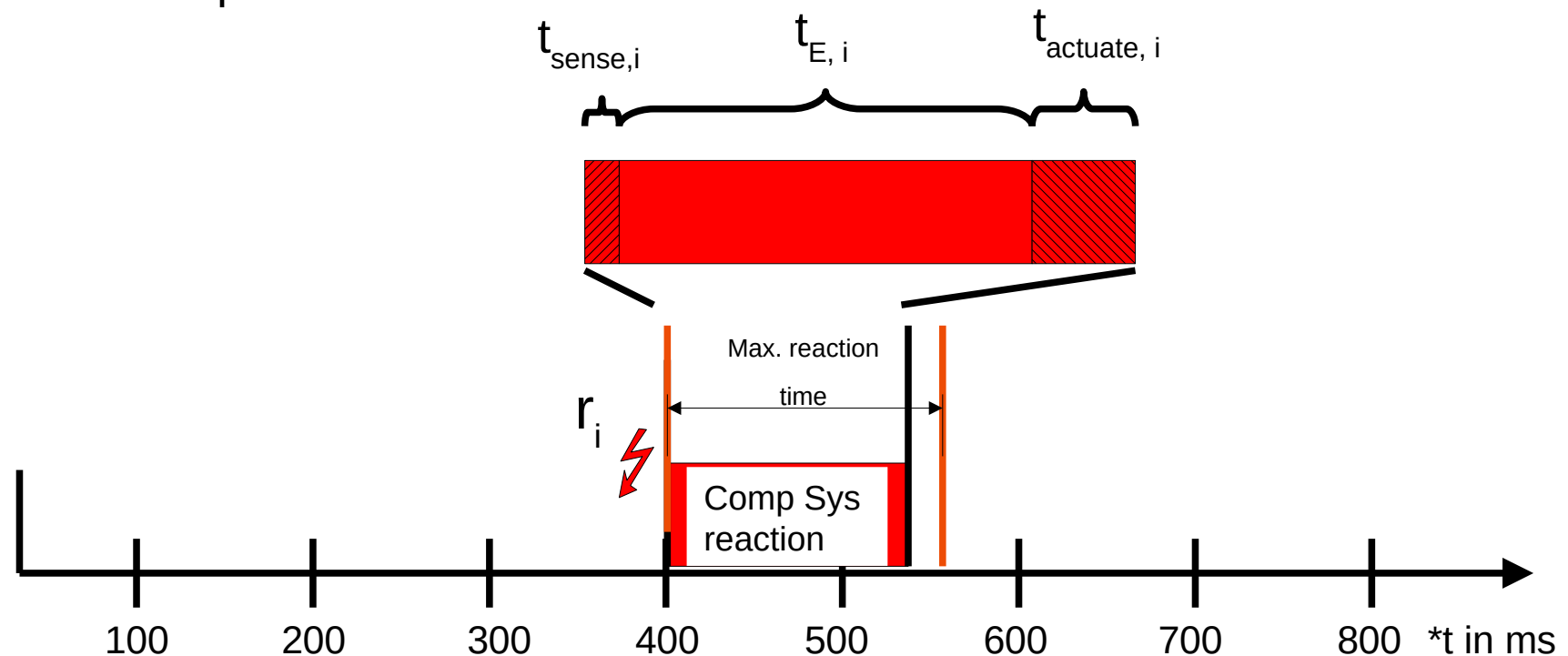
Reaction  
on  
request  $r_i$

# Reaction Time = Sense Time + Exec. Time + Act. Time



TH Aschaffenburg  
university of applied sciences

Reaction time is split into physical effects on sensor and actuator side and the computational execution time



$t_{\text{sense},i}$  time frame needed to sense i-th request  $r_i$

$t_{E,i}$  time frame for executing control algorithm on i-th request

$t_{\text{actuate},i}$  time frame needed to actuate on i-th request

\* numbers  
are examples

# Soft Real Time and Hard Real Time Systems

- Extending max. reaction in some tolerances allowed
  - Delays cause acceptable costs
  - Drop in quality and user acceptance
  - Typical applications: Telephone system, vending machine, desktop computers, multimedia applications such as audio/video streaming
- Deadlines must be strictly met  
 $\text{exec time} < \text{max exec time}$
  - Extending the deadline is not accepted under any circumstances
  - Disastrous consequences (live danger) and high cost
  - Calc. Results are useless if they are achieved too late
  - Application examples:  
Production robots, (nuclear) plant control, Vehicle Control System (ABS, ESP etc. ), aircraft flight control, monitoring systems of an intensive care unit, real time trading systems (shares, money transfer)



# Real Time Operating Systems (RTOS) for Hard Real Time Applications - Examples



TH Aschaffenburg  
university of applied sciences

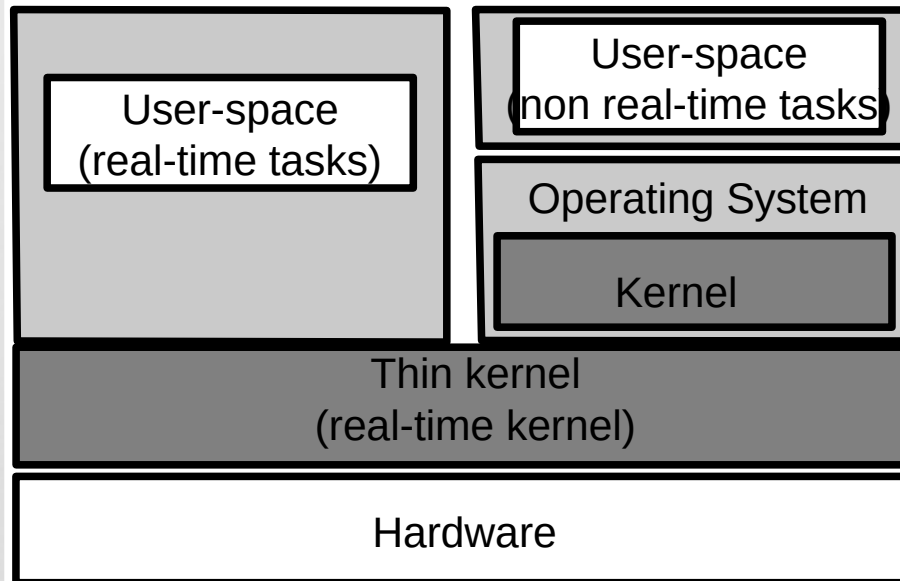
- **RTLinux – extension to main line kernel of Linux**
- **Some commercially available:**
  - QNX
  - VxWorks
  - LynxOS
  - ...

# Architectures of RTOSs

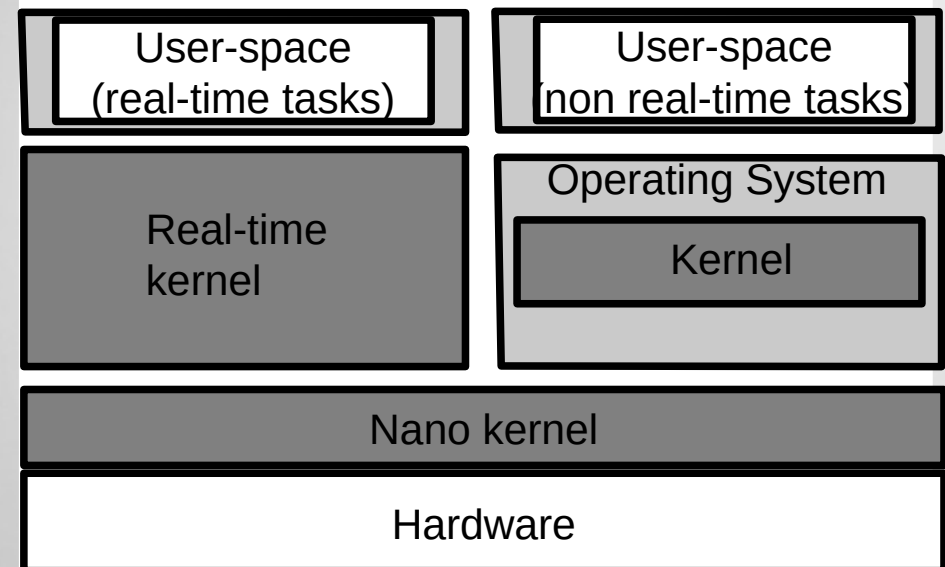


TH Aschaffenburg  
university of applied sciences

## Thin Kernel



## Nano Kernel



# What are the Main Features of an OS?

## Recap the Fundamentals



TH Aschaffenburg  
university of applied sciences

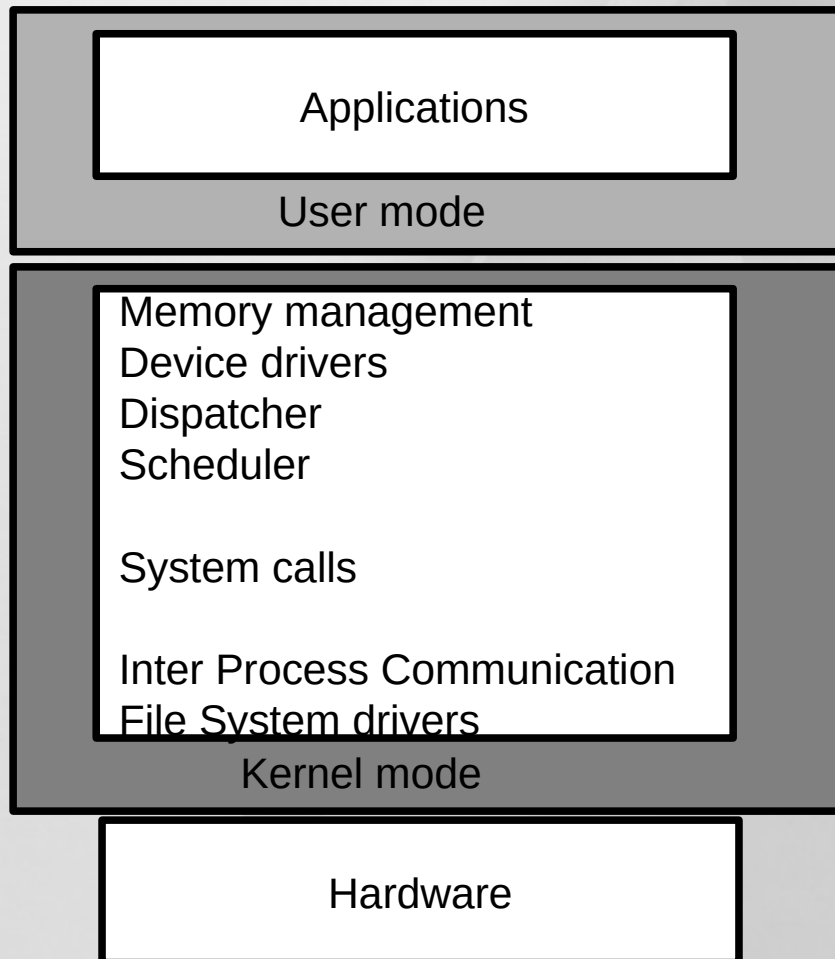
### Management of:

- Users and Groups, security and authorization, User Interface
- Data
- Memory
- Processes/Tasks, synchronisation and cooperation (include communication)
- Hardware, all other than CPU and memory

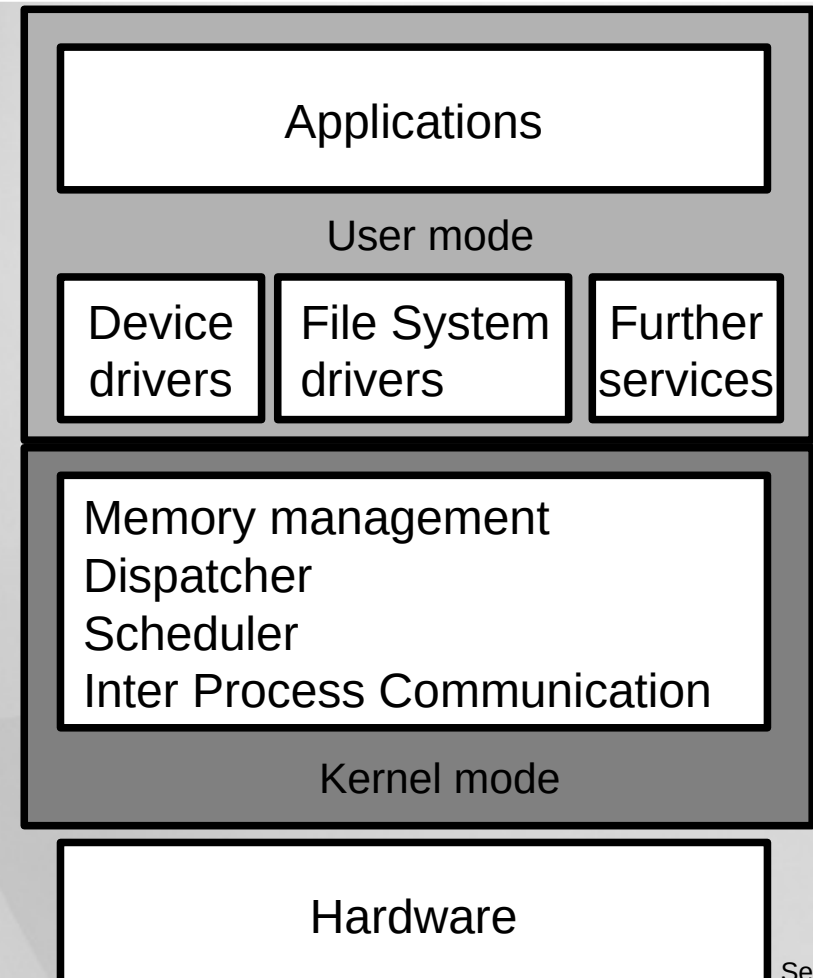
# Terminology (more details later on)

- **User Mode**
  - Unprivileged mode
  - User programs
    - Interfacing to Operating System
    - Isolated virtual memory space
    - Rights management of OS
- **Kernel Mode**
  - Privileged mode
  - Full access to hardware
  - Access to all physical memories

- **Monolithic Kernel**



- **Microkernel**



# Pros and Cons of the Kernel Architectures

## Monolithic Kernel

- **Advantages**

Single source for all parts of the OS  
wrt. update / functionality

- **Drawbacks**

If some driver or part is erroneous, the whole kernel may crash

Security risk from the driver perspective

Unneeded parts (for different target hardware) may blow the whole kernel

## Microkernel

- **Advantages**

Modularity for different hardware driver

Security might be enhanced

Error in one driver does not crash the whole system

- **Drawbacks**

Fixing interface between kernel and driver parts could be difficult

Efficiency is lower compared to monolithic kernel

# Hybrid Kernel aka Macrokernel



TH Aschaffenburg  
university of applied sciences

